

API для публикации события

Публикацию события можно производить в исходном коде как на C++, так и на Python. В обоих случаях используется метод Publish(), который публикует событие в Брокер. Метод Publish() принимает экземпляр класса Event, аргументы конструктора и методы которого представлены в таблице ниже. Для именования аргументов используйте латинские буквы нижнего регистра.

Аргумент в Python	Аналог в C++	Тип	Описание
Серверные события			
name	1-й аргумент конструктора	Строка	Имя (идентификатор) события и канала, в котором оно публикуется. При выборе имени нужно руководствоваться правилами именования события .
visibility	2-й аргумент конструктора	event::Visibility (C++) Enum event.Visibility (Python)	Получатель сообщения события. • evCLIENT_ONLY — только клиентские приложения. • evSERVER_ONLY — только сервисы БЛ. • evALL — клиентские приложения и сервисы БЛ.
payload	SetPayload	Объект любого платформенного типа, сериализуемого в JSON: string, int, double, DateTime, Record, RecordSet и HashTable	Объект события. Приходит подписчикам в теле сообщения события.
policy	SetPolicy	sbis::PublicationPolicy (C++) Enum sbis.PublicationPolicy (Python)	Политика публикации события. • ppIMMEDIATELY — сообщение отсылается сразу. • ppON_TRANSACTION_COMMIT (значение по умолчанию) — сообщение отсылается при успешном завершении текущей транзакции. Если транзакция отсутствует, то публикуется сразу. • ppON_REQUEST_FINISH — сообщение отсылается при завершении обработки входящего запроса самого первого метода, пришедшего на сервер.
priority	SetPriority	event::Priority (C++) Enum event.Priority (Python)	Приоритет события .

			• evPRIORITY_NO — без приоритета. • evPRIORITY_LOW — низкий приоритет. • evPRIORITY_MEDIUM — средний приоритет. • evPRIORITY_HIGH — высокий приоритет.
agg_key	SetAggKey	Строка или event::AggregationKey (C++) event.AggregationKey (Python)	Ключ агрегации события. Когда значение параметра не задано или передано как пустая строка, событие публикуется по обычной схеме оповещения . При других значениях параметра публикация выполняется через Сервис агрегации событий .
Клиентские события			
application	SetApplication	Строка	Приложение, в адрес которого публикуется событие. Используется для публикации события в адрес приложения. Для именования значений аргумента используйте латинские буквы нижнего регистра, цифры и точку.  Опубликованное событие в адрес приложения будет доставлено только тем, кто явно подписался на это событие и это приложение.
applications	SetApplications	Массив строк	Приложения, в адрес которых публикуется событие. Используется для публикации события в адрес приложения.  Опубликованное событие в адрес приложений будет доставлено только тем, кто явно подписался на это событие и хотя бы одно приложение из списка приложений.
global, is_global	SetGlobal	Boolean для Python Без аргументов для C++	Сообщение события отсылается всем клиентским приложениям. Используется только в

			особых случаях, т.к. при большом потоке таких сообщений сильно нагружается RabbitMQ. Чтобы использовать этот уровень изоляции, требуется согласование с Головановым К.А.
users	SetUser, SetUsers	Массив целых чисел (Число – в случае SetUser в C++)	Получателем события устанавливаются указанные пользователи .
clients	SetClient, SetClients	Массив целых чисел (Число – в случае SetClient в C++)	Получателем события устанавливаются указанные клиенты. При публикации события в адрес клиента следует указывать приложение (приложения)/ канализированность!  В случае необходимости публикации события в адрес клиента без указания приложения (приложений)/ канализированности, требуется согласование Голованова К.А. или Смирнова И.В.
persons	SetPerson, SetPersons	Массив строк (строка – в случае SetPerson в C++)	Получателем события устанавливаются указанные персоны .
sites	SetSites	Массив строк	DEPRECATED Сайты больше не учитываются при доставке событий
channelled, is_channelled	SetChannelled	Boolean для Python. Без аргументов для C++.	DEPRECATED Создано для обратной совместимости со старой схемой канализированных событий, используйте application.

Требования к программному окружению

Для публикации события из исходного кода C++:

1. В состав сервиса приложения включите сервисный модуль EventSubscription.
2. В свойствах вашего сервисного модуля добавьте зависимость от модуля EventSubscription.
3. В пользовательской библиотеке в файле CMakeLists.txt добавьте sbis_add_sdk_dependency("sbis-event-subscription300").
4. В исходном файле подключите заголовочный файл sbis-event-subscription/publish.hpp.

Для публикации события из исходного кода Python:

1. Включите в состав сервиса приложения сервисные модули EventSubscription и EventSubscription-Py.

2. В свойствах вашего сервисного модуля добавьте зависимость от модулей EventSubscription и EventSubscription-Py.
3. В исходном файле подключите модуль event.